

Machine Translation on Sequence to Sequence

KyungTae Lim



	TYU division			FRT division		
GHT	254	550	254	274	154	415
RDW	650	320	754	273	825	154
TRG	241	450	144	364	954	174
RTG	254	650	874	657	125	274
WTF	754	154	154	754	754	754
WPT	154	754	754	241	741	245

Contents

01 Vanishing Gradient

02 LSTM

03 Machine Translation on Seq2Seq

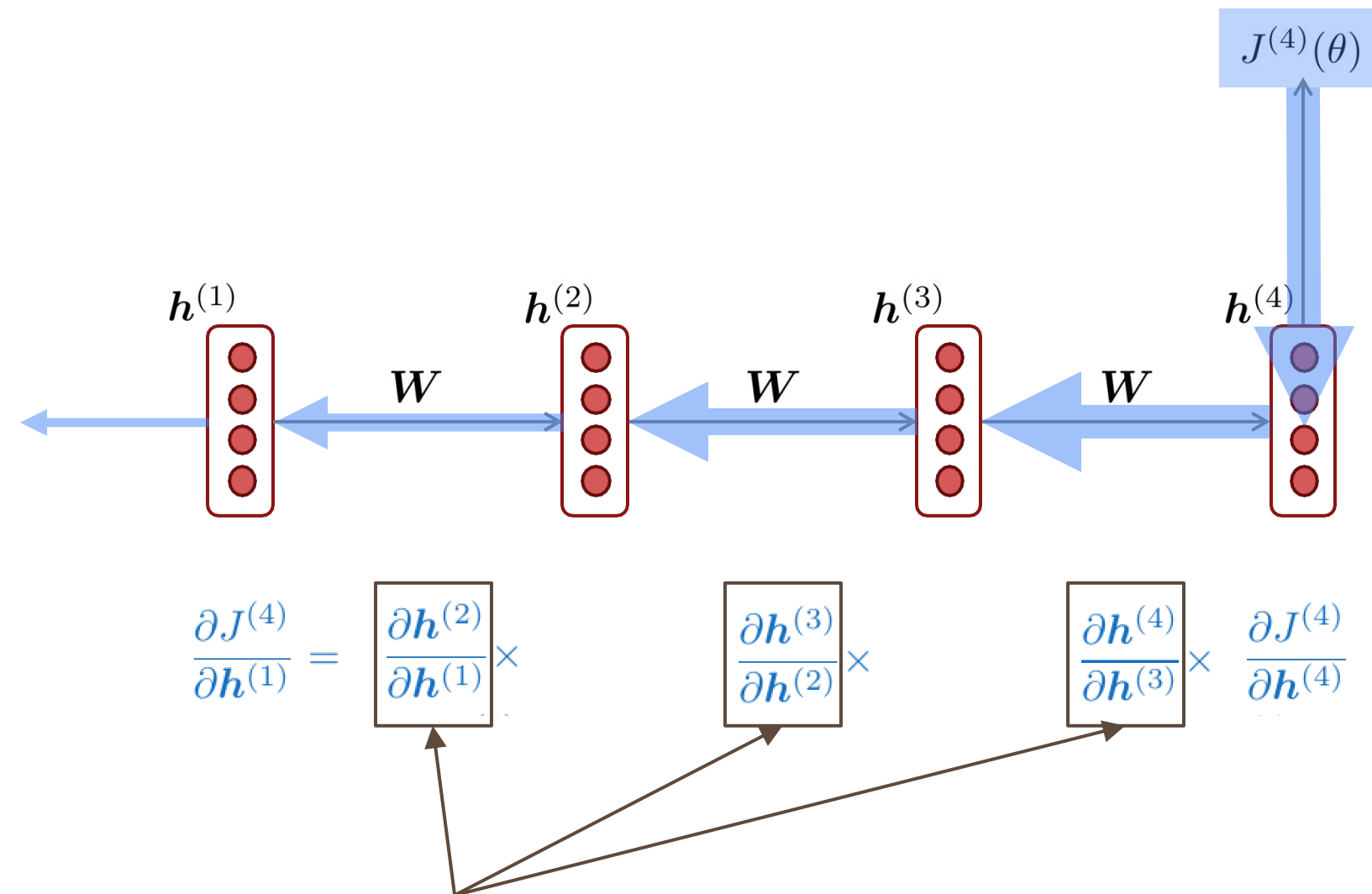


We are talking about

- **Language Model**: A system that predicts the next word
- **Recurrent Neural Network**: A family of neural networks that:
 - Take sequential input of any length; apply the same weights on each step
 - Can optionally produce output on each step
- **Recurrent Neural Network $\neq$ Language Model**
 - RNNs can be used for many other things (see later)
- **Language Modeling** is a traditional subcomponent of many NLP tasks, all those involving generating text or estimating the probability of text:
 - Now everything in NLP is being rebuilt upon Language Modeling: GPT is an LM!

We are talking about

- **Vanishing Gradient** The gradient decays exponentially through repeated multiplication, making early states almost untrainable.

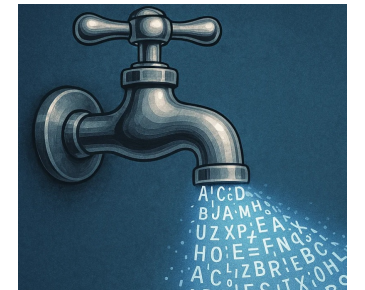


What happens if these are small?

Vanishing gradient problem: When these are small, the gradient signal gets smaller and smaller as it backpropagates further

-
- The diagram illustrates the forward and backward passes of a recurrent neural network (RNN) unrolled over four time steps. The forward pass (blue arrows) flows from right to left, starting with input $h^{(4)}$ (a vertical vector of four red circles) which is multiplied by weight matrix W to produce $h^{(3)}$. This process repeats to produce $h^{(2)}$ and then $h^{(1)}$. A final blue arrow points left from $h^{(1)}$. The backward pass (brown arrows) flows from left to right, starting from the loss $J^{(4)}(\theta)$ (a blue box) which is connected to $h^{(4)}$. The backward pass calculates gradients $\frac{\partial h^{(4)}}{\partial h^{(3)}}$, $\frac{\partial h^{(3)}}{\partial h^{(2)}}$, and $\frac{\partial h^{(2)}}{\partial h^{(1)}}$ (each in a brown box), which are then combined to find $\frac{\partial J^{(4)}}{\partial h^{(1)}}$ (in a blue box). The final result is shown as $\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$.

A green cartoon frog with large, white eyes with black pupils. It has a wide, orange-brown mouth with a slight smile. The frog is wearing a blue shirt. Its right hand is raised near its face. The background is a solid light blue.



What happens if these are small?

Vanishing gradient problem: When these are small, the gradient signal gets smaller and smaller as it backpropagates further

Contents

02 LSTM

03 Machine Translation on Seq2Seq



Long Short-Term Memory (LSTMs)

- LSTM prevents vanishing gradients by preserving information through an additive path rather than repeated multiplication.
 - To do this LSTM has a **special memory(=notepad)** system named cell state
 - The cell stores **long-term information**

RNN (Multiplication)

$$h_t = \tanh(W h_{t-1})$$

LSTM (Copy)

$$C_t \approx C_{t-1}$$

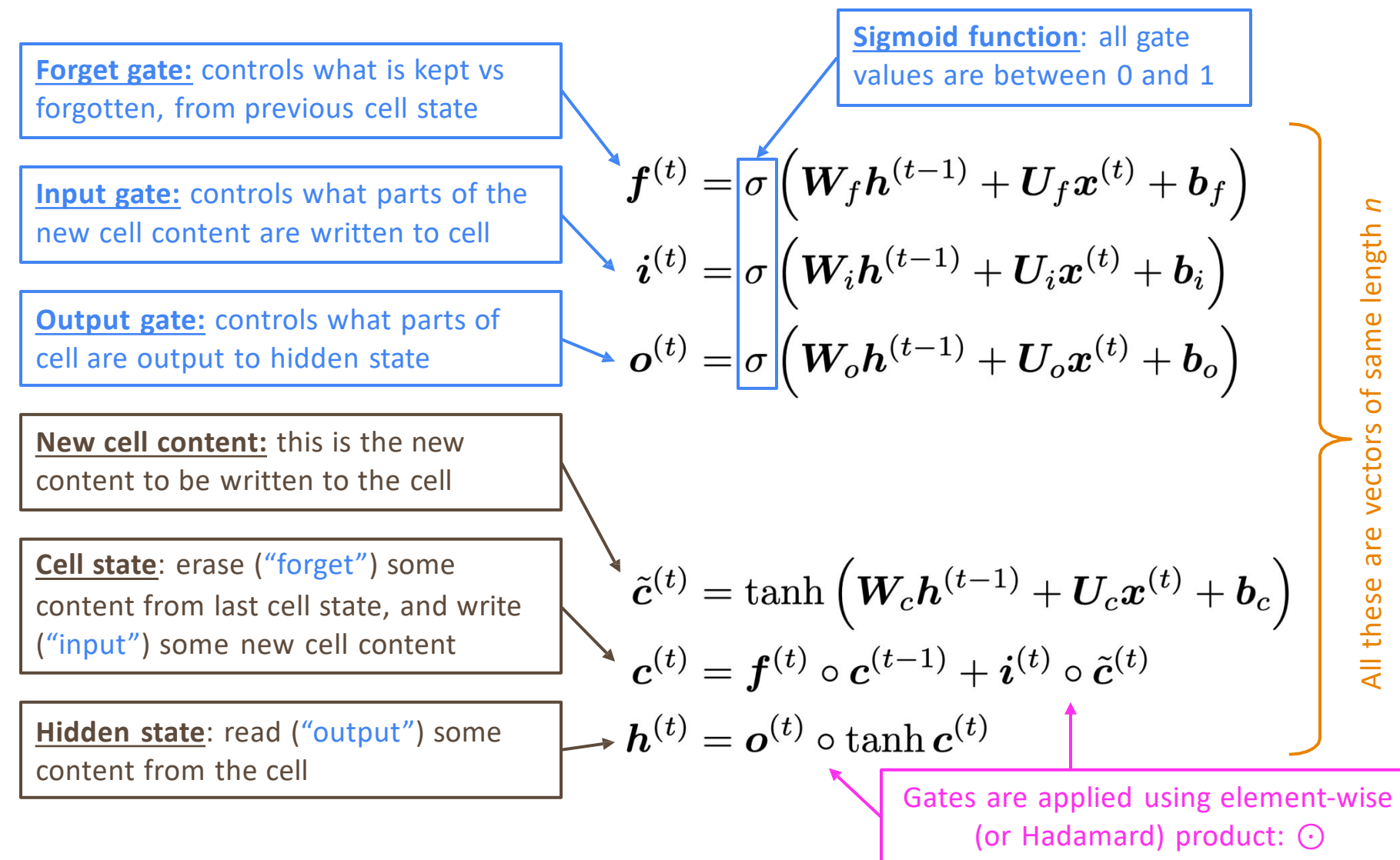


Long Short-Term Memory (LSTMs)

- On step t , there is a **hidden state** $\mathbf{h}^{(t)}$ and a **cell state** $\mathbf{c}^{(t)}$
 - Both are vectors length n
 - The cell stores **long-term information**
 - The LSTM can **read**, **erase**, and **write** information from the cell
 - The cell becomes conceptually rather like RAM in a computer
- The selection of which information is erased/written/read is controlled by three corresponding **gates**
 - The gates are also vectors of length n
 - On each timestep, each element of the gates can be **open** (1), **closed** (0), or somewhere in-between
 - The gates are **dynamic**: their value is computed based on the current context

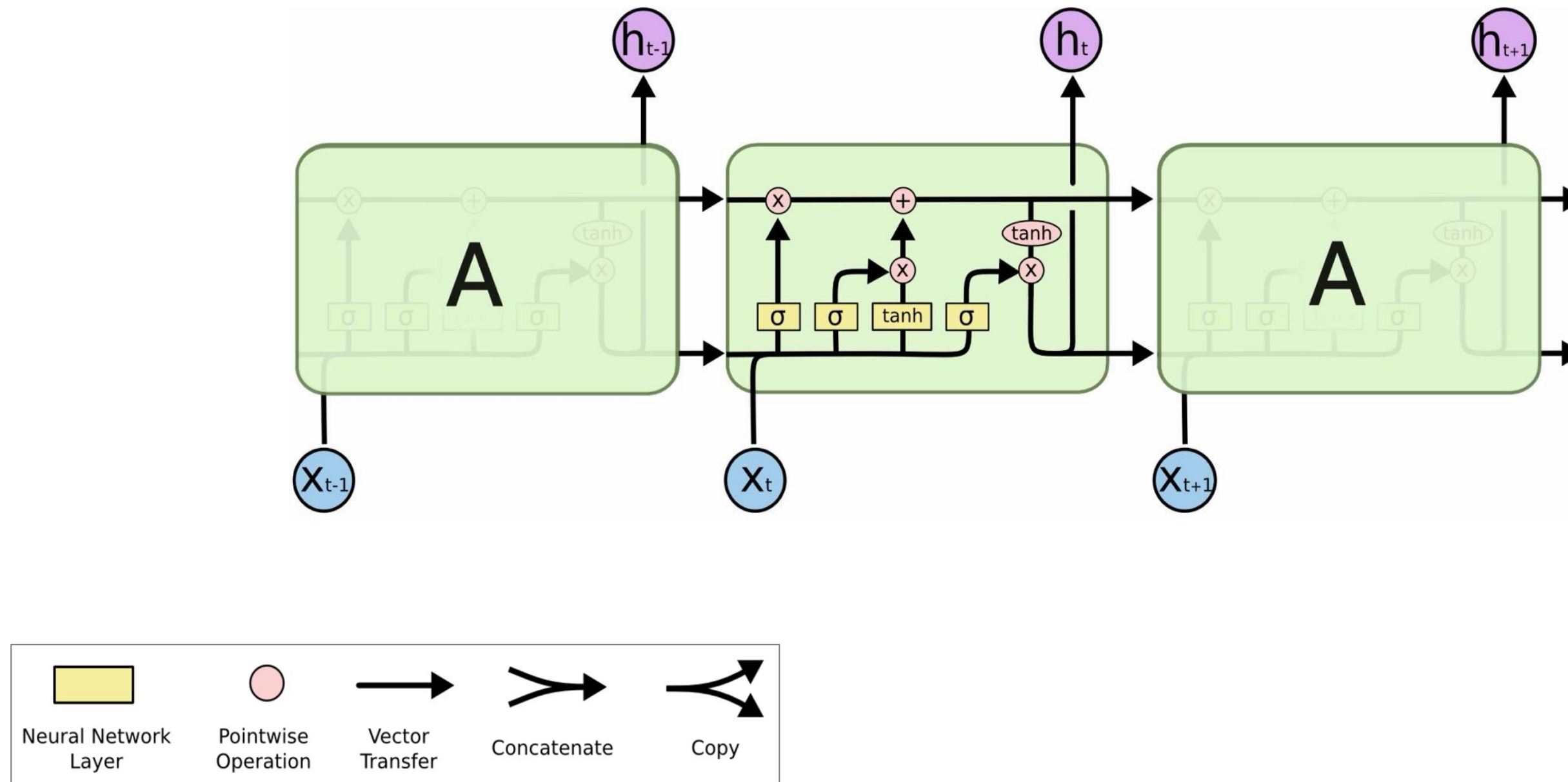
Long Short-Term Memory (LSTMs)

We have a sequence of inputs $x^{(t)}$, and we will compute a sequence of hidden states $h^{(t)}$ and cell states $c^{(t)}$. On timestep t :



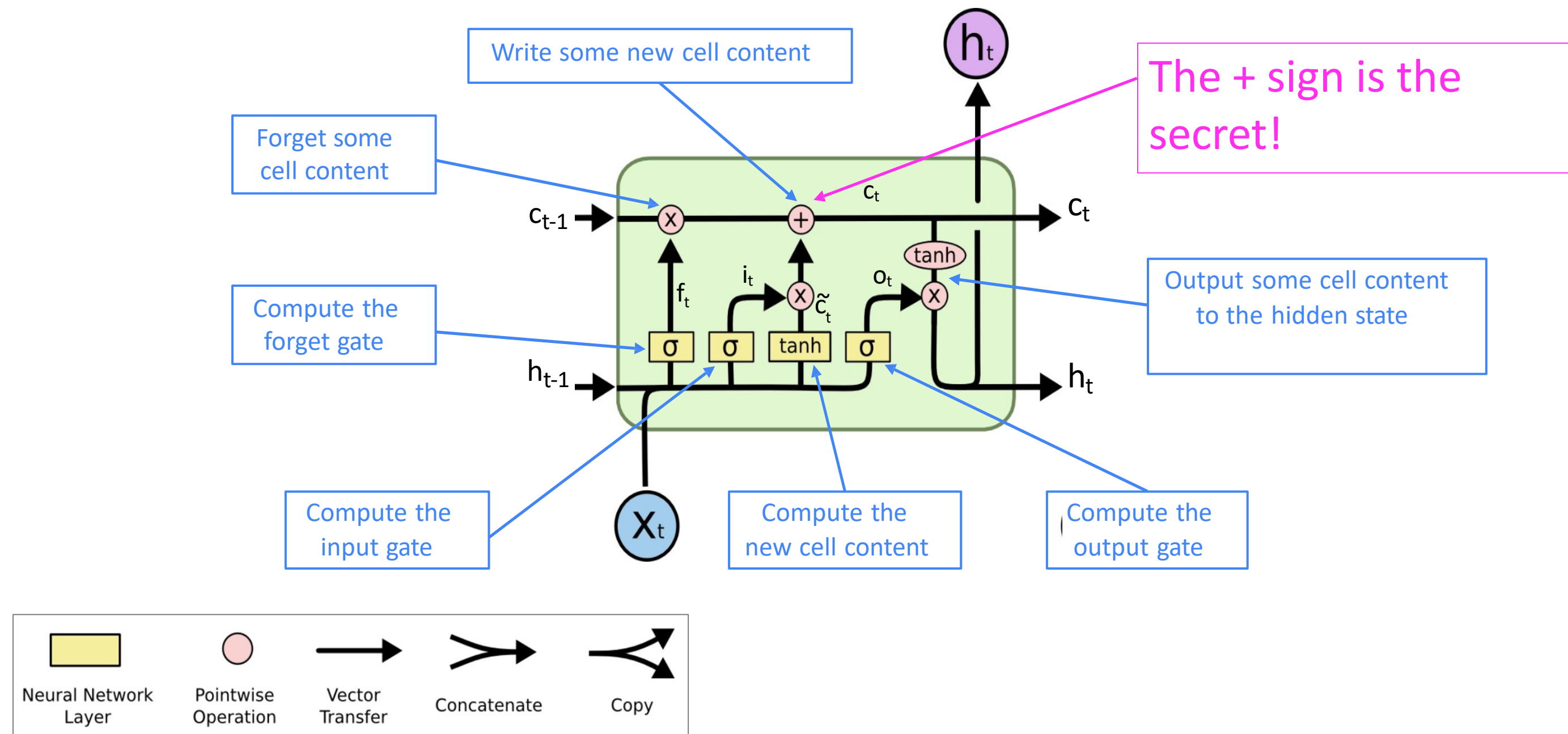
Long Short-Term Memory (LSTMs)

You can think of the LSTM equations visually like this:



Long Short-Term Memory (LSTMs)

You can think of the LSTM equations visually like this:



Long Short-Term Memory (LSTMs)

- The LSTM architecture makes it **much easier** for an RNN to **preserve information over many timesteps**
 - e.g., if the forget gate is set to 1 for a cell dimension and the input gate set to 0, then the information of that cell is preserved indefinitely.
 - In contrast, it's harder for a vanilla RNN to learn a recurrent weight matrix W_h that preserves info in the hidden state
 - In practice, you get about 100 timesteps rather than about 7 in case of RNN
- However, there are alternative ways of creating more direct and linear pass-through connections in models for long distance dependencies

Long Short-Term Memory (LSTMs)

- In 2014–2018, LSTMs started achieving state-of-the-art results
 - Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
 - LSTMs became the dominant approach for most NLP tasks
- Recently (2019–2024), Transformers have become dominant for all tasks
 - For example, in WMT (a Machine Translation conference + competition):
 - In WMT 2014, there were 0 neural machine translation systems (!)
 - In WMT 2016, the summary report contains “RNN” 44 times (and these systems won)
 - In WMT 2019: “RNN” 7 times, “Transformer” 105 times
- Now, ‘State space models’ (RNN++) are making a comeback
 - Recurrent Weighted Key-Value: RWKV (for enhancing inference efficiency)
 - Mamba (for enhancing modeling long sequences)

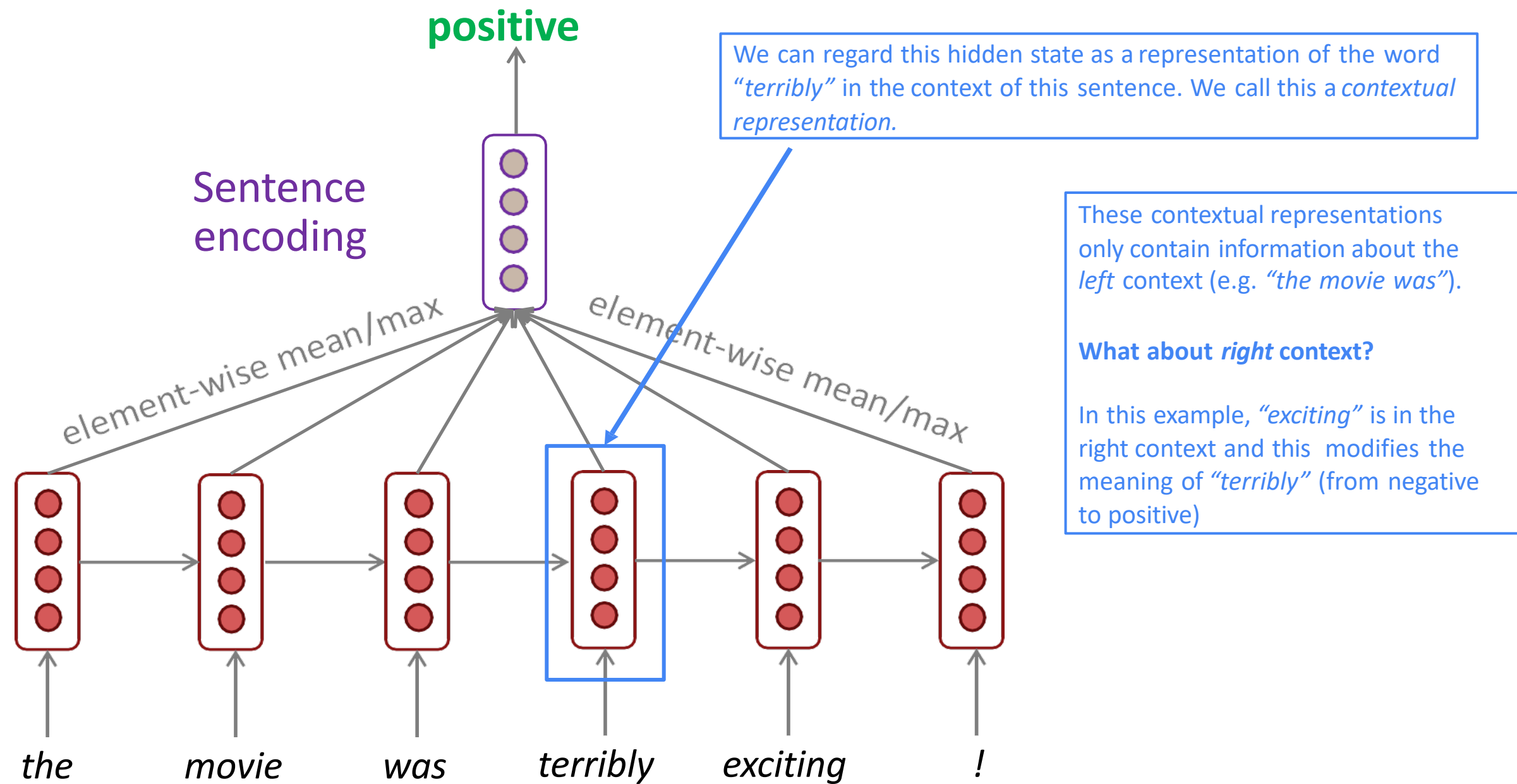
Source: "Findings of the 2016 Conference on Machine Translation (WMT16)", Bojar et al. 2016, <http://www.statmt.org/wmt16/pdf/W16-2301.pdf>

Source: "Findings of the 2018 Conference on Machine Translation (WMT18)", Bojar et al. 2018, <http://www.statmt.org/wmt18/pdf/WMT028.pdf>

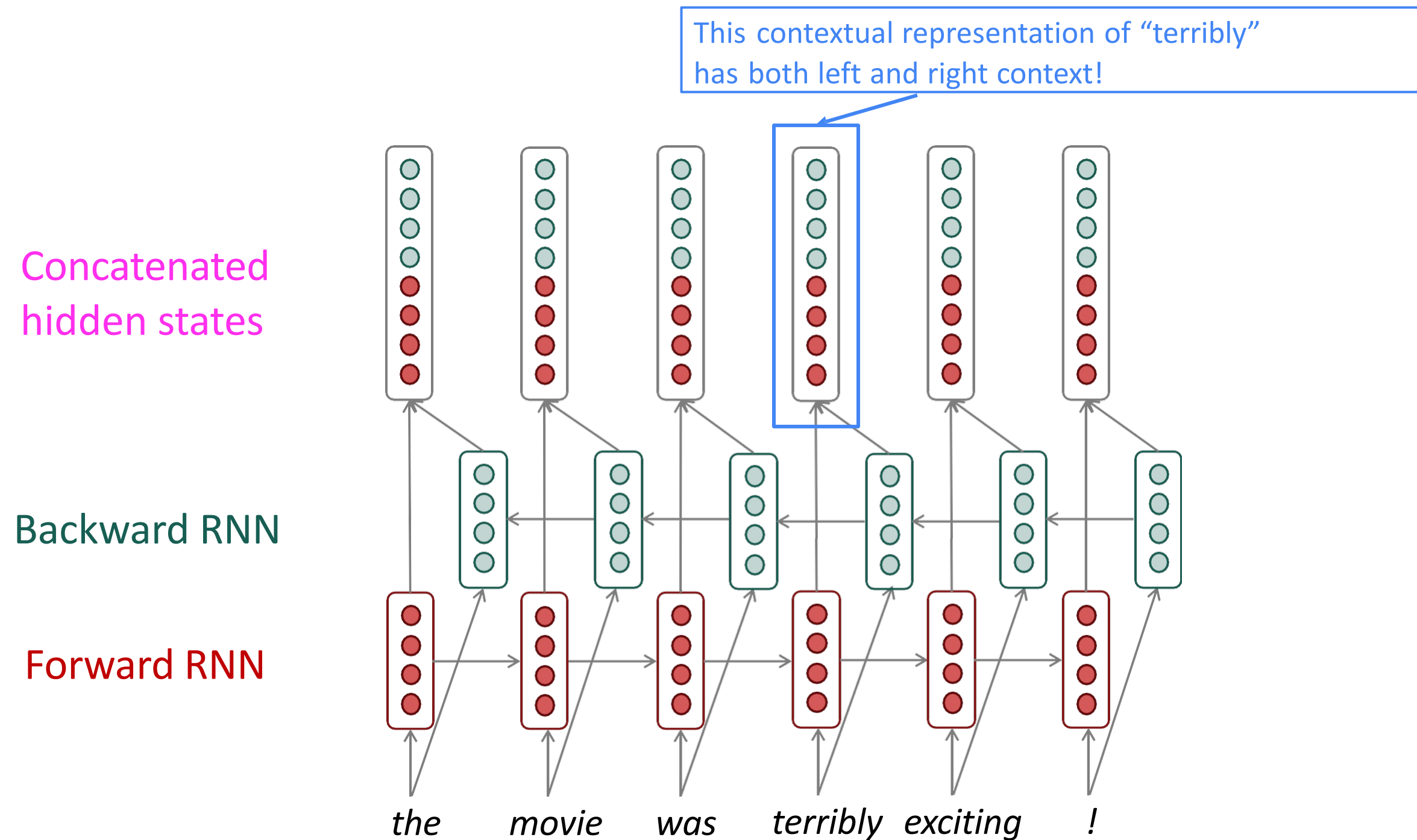
Source: "Findings of the 2019 Conference on Machine Translation (WMT19)", Barrault et al. 2019, <http://www.statmt.org/wmt18/pdf/WMT028.pdf>

Bidirectional RNNs

Task: Sentiment Classification



Bidirectional RNNs



Bidirectional RNNs

This is a general notation to mean “compute one forward step of the RNN” – it could be a simple RNN or LSTM computation.

Forward RNN $\vec{h}^{(t)} = \text{RNN}_{\text{FW}}(\vec{h}^{(t-1)}, \mathbf{x}^{(t)})$

Backward RNN $\overleftarrow{h}^{(t)} = \text{RNN}_{\text{BW}}(\overleftarrow{h}^{(t+1)}, \mathbf{x}^{(t)})$

Generally, these two RNNs have separate weights

Concatenated hidden states $\mathbf{h}^{(t)} = [\vec{h}^{(t)}; \overleftarrow{h}^{(t)}]$

We regard this as “the hidden state” of a bidirectional RNN. This is what we pass on to the next parts of the network.

Bidirectional RNNs

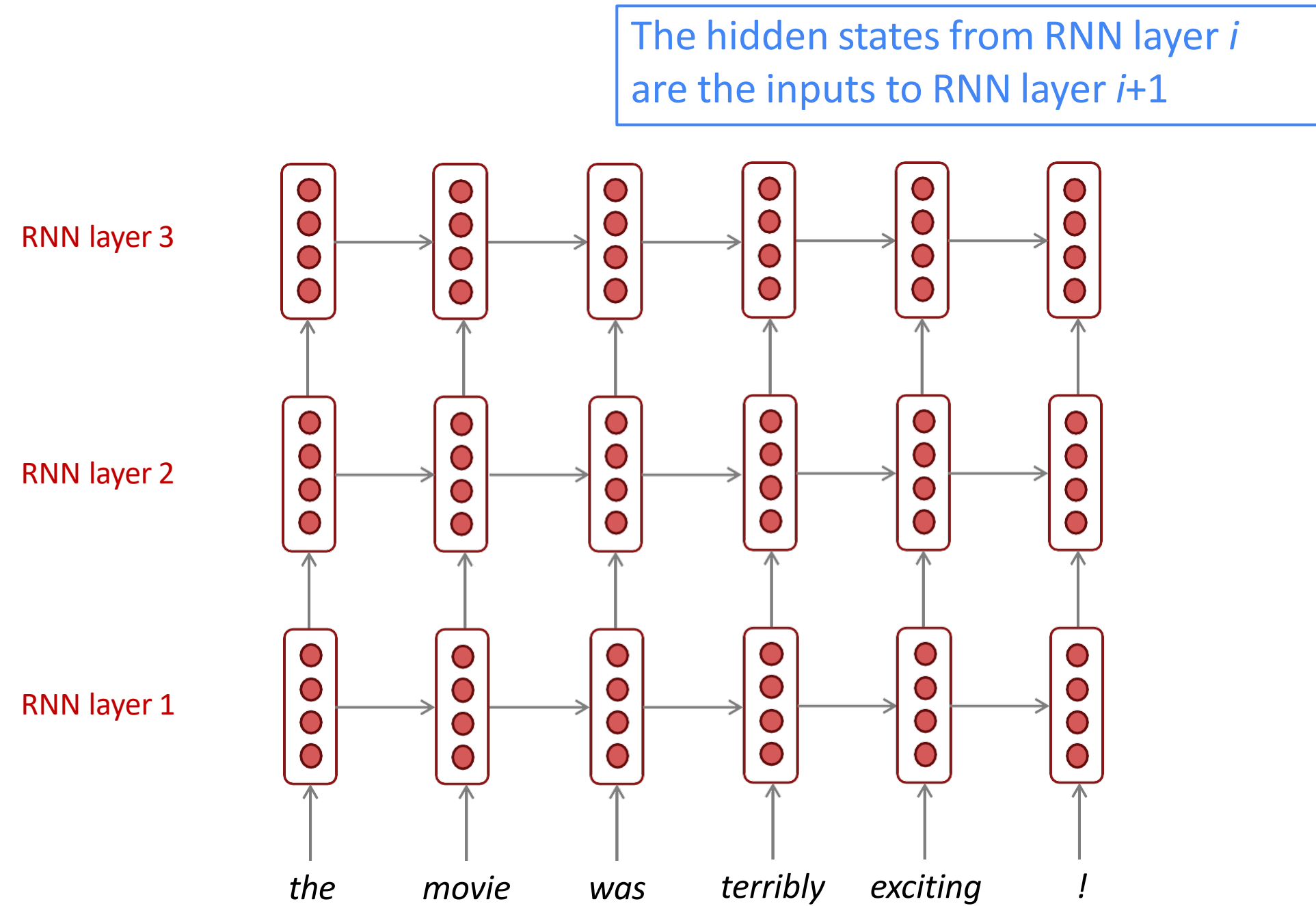
- Note: bidirectional RNNs are only applicable if you have access to the **entire input sequence**
 - They are **not** applicable to Language Modeling, because in LM you *only* have left context available.
- If you do have entire input sequence (e.g., any kind of encoding), **bidirectionality is powerful** (you should use it by default).
- For example, **BERT** (**Bidirectional** Encoder Representations from Transformers) is a powerful pretrained contextual representation system **built on bidirectionality**.
 - You will learn more about **transformers**, including BERT, in a couple of weeks!

Multi-layer RNNs

- RNNs are already “deep” on one dimension (they unroll over many timesteps)
- We can also make them “deep” in another dimension by **applying multiple RNNs** – this is a multi-layer RNN.
- This allows the network to compute **more complex representations**
 - The **lower RNNs** should **compute lower-level features** and the **higher RNNs** should compute **higher-level features**.
- Multi-layer RNNs are also called ***stacked RNNs***.



Multi-layer RNNs



Contents

03 Machine Translation on Seq2Seq



Machine Translation

Machine Translation (MT) is the task of translating a sentence x from one language (the **source language**) to a sentence y in another language (the **target language**).

< google's collections 49 🔍

Gemma 4 >

 google/gemma-4-31B-it
Image-Text-to-Text • 33B • 679k ⚡

 google/gemma-4-31B
Image-Text-to-Text • 33B • 39.6k ↶

 google/gemma-4-26B-A4B-it
Image-Text-to-Text • 27B • 477k ⚡

 google/gemma-4-26B-A4B

TranslateGemma >

 google/translategemma-27b-it
Image-Text-to-Text • U... • 12.2k • 356

 google/translategemma-4b-it
Image-Text-to-Text • U... • 71.2k • 727

 google/translategemma-12b-it
Image-Text-to-Text • U... • 68.6k • 285

$x:$ *L'homme est né libre, et partout il est dans les fers*



$y:$ *Man is born free, but everywhere he is in chains*

– Rousseau

History of Machine Translation

- Machine translation research began in the **early 1950s** on machines less powerful than high school calculators (before term “A.I.” coined!)
- Concurrent with foundational work on automata, formal languages, probabilities, and information theory
- MT heavily funded by military, but basically just simple rule-based systems doing word substitution
- Human language is more complicated than that, and varies more across languages!
- Little understanding of natural language syntax, semantics, pragmatics
- Problem soon appeared intractable

History of Machine Translation



1 minute video showing 1954 MT:

<https://youtu.be/8ZtdVUB007A?si=IQr0KbbCqgxiCJcU>

History of Machine Translation (1990s ~ 2010s)

- Core idea: Learn a **probabilistic model** from **data**
- Suppose we're translating French $\rightarrow$ English.
- We want to find **best English sentence** y , given **French sentence** x

$$\operatorname{argmax}_y P(y|x)$$

- Use Bayes Rule to break this down into **two components** to be learned separately:

$$= \operatorname{argmax}_y \underbrace{P(x|y)}_{\text{Translation Model}} \underbrace{P(y)}_{\text{Language Model}}$$

Translation Model

Models how words and phrases
should be translated (*fidelity*).
Learned from parallel data.

Language Model

Models how to write good
English (*fluency*).
Learned from monolingual data.

History of Machine Translation (1990s ~ 2010s)

- SMT was a huge research field
- The best systems were extremely complex
 - Hundreds of important details
- Systems had many separately-designed subcomponents
 - Lots of feature engineering
 - Need to design features to capture particular language phenomena
 - Required compiling and maintaining extra resources
 - Like tables of equivalent phrases
 - Lots of human effort to maintain
 - Repeated effort for each language pair!

History of Machine Translation (2010s ~)

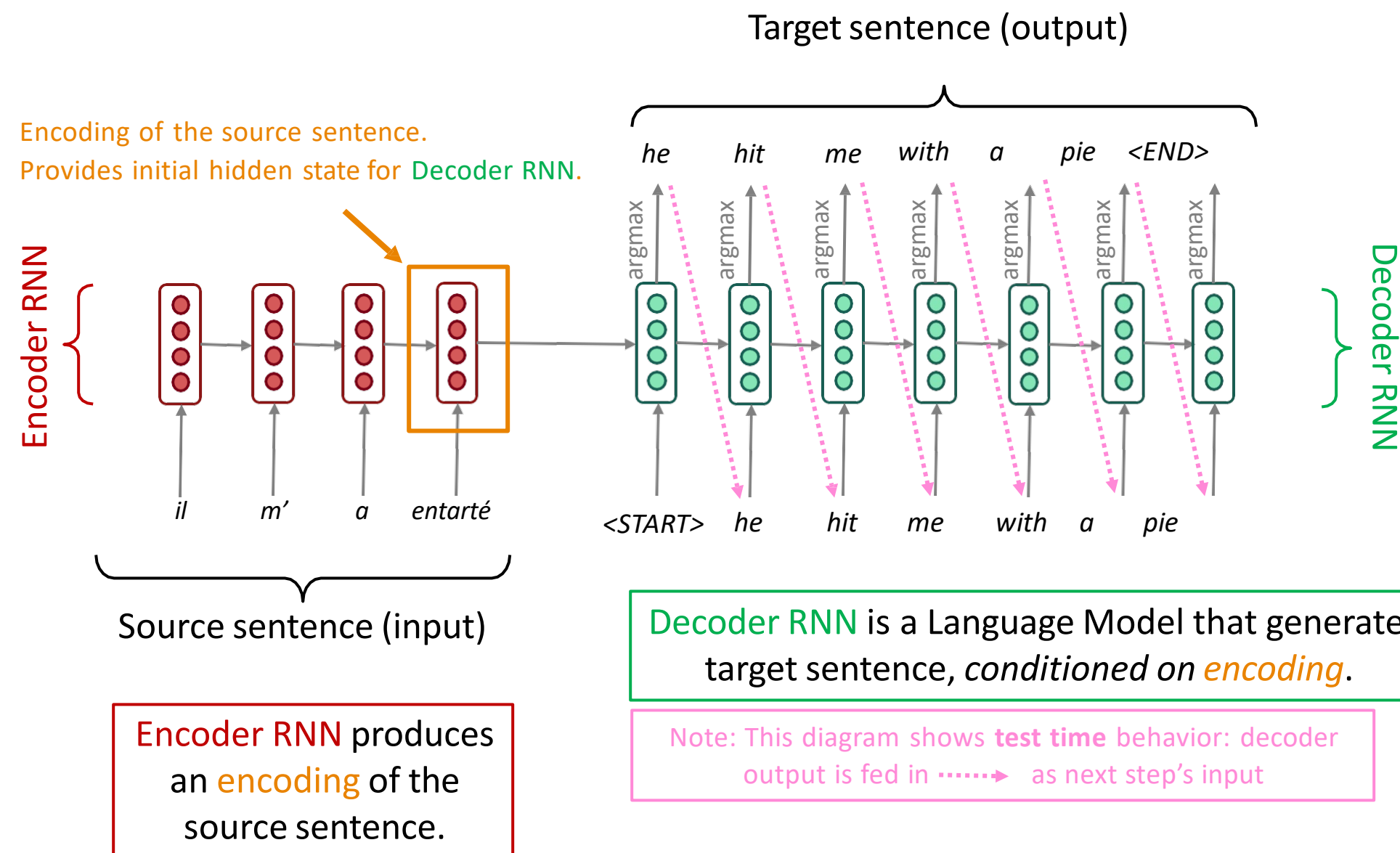
Neural Machine Translation went from a fringe research attempt in **2014** to the leading standard method in **2016**

- **2014**: First seq2seq paper published [Sutskever et al. 2014]
- **2016**: Google Translate switches from SMT to NMT – and by 2018 everyone has



- This is amazing!
 - **SMT** systems, built by hundreds of engineers over many years, outperformed by NMT systems trained by small groups of engineers in a few months

The sequence-to-sequence model



The sequence-to-sequence model

- The general notion here is an **encoder-decoder** model
 - One neural network takes input and produces a neural representation
 - Another network produces output based on that neural representation
 - If the input and output are sequences, we call it a seq2seq model
- Sequence-to-sequence is useful for *more than just MT*
- Many NLP tasks can be phrased as sequence-to-sequence:
 - **Summarization** (long text → short text)
 - **Dialogue** (previous utterances → next utterance)
 - **Parsing** (input text → output parse as sequence)
 - **Code generation** (natural language → Python code)

Translation on Seq2seq

- The **sequence-to-sequence** model is an example of a **Conditional Language Model**
 - **Language Model** because the decoder is predicting the next word of the target sentence y
 - **Conditional** because its predictions are *also* conditioned on the source sentence x

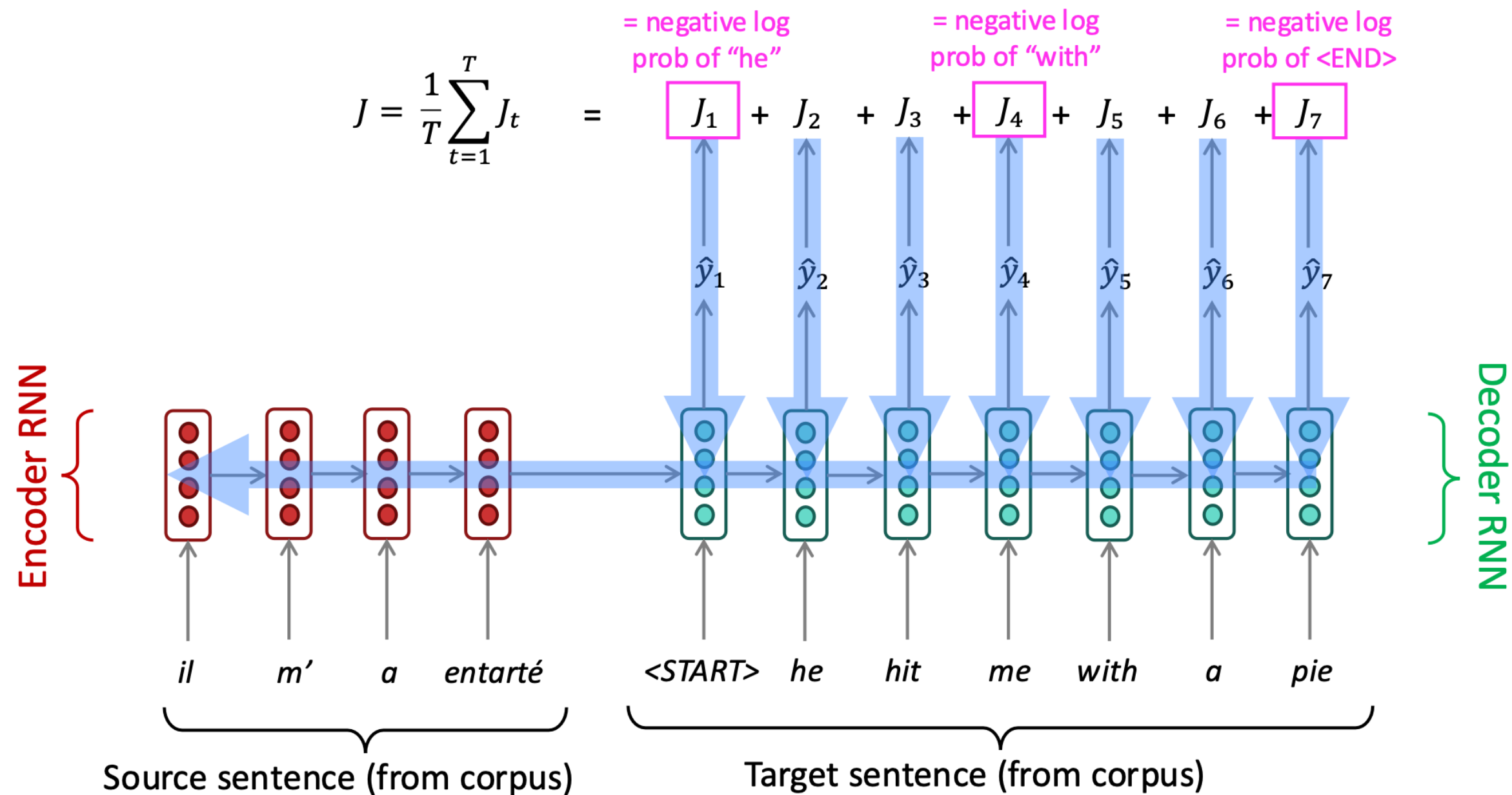
- NMT directly calculates $P(y|x)$

$$P(y|x) = P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots P(y_T|y_1, \dots, y_{T-1}, x)$$

Probability of next target word, given
target words so far and source sentence x

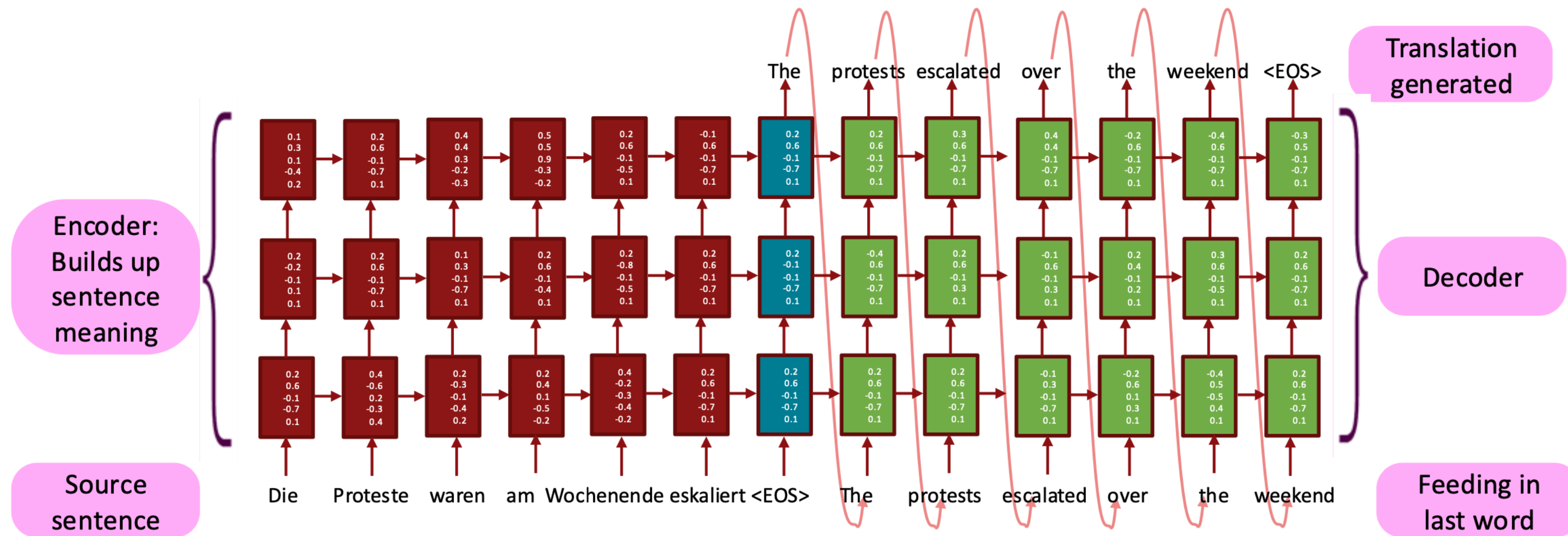
- **Question:** How to train an NMT system?
- **(Easy) Answer:** Get a big parallel corpus...
 - But there is now exciting work on “unsupervised NMT”, data augmentation, etc.

Translation on Seq2seq



Seq2seq is optimized as a **single system**. Backpropagation operates “*end-to-end*”.

Multi-layer encoder-decoder machine translation



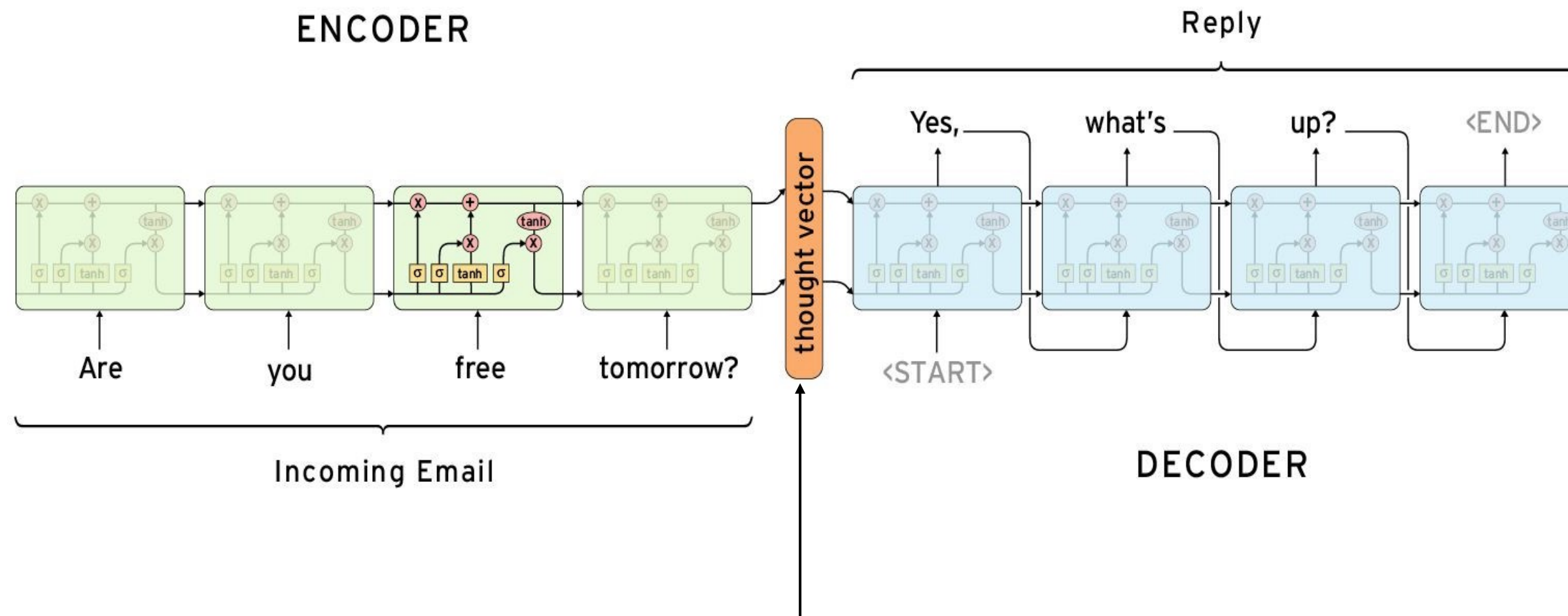
Contents

03 Chatbot on Seq2seq



The seq2seq Chatbot using LSTM

- What is the difference between general RNN model and Seq2Seq model?
- We just need **two different RNN** structures to build seq2seq!



The encoder's first hidden is fed in to the first hidden state of Decoder

The seq2seq Chatbot using LSTM

- **Main Function**

```
4  import random
5  import torch
6  import torch.nn as nn
7  import torch.optim as optim
```

```
SOURCE_MAX_LENGTH = 10
TARGET_MAX_LENGTH = 12
load_pairs, load_source_vocab, load_target_vocab = preprocess(raw, SOURCE_MAX_LENGTH, TARGET_MAX_LENGTH)
print(random.choice(load_pairs))
```

```
enc_hidden_size = 16
dec_hidden_size = enc_hidden_size
enc = Encoder(load_source_vocab.n_vocab, enc_hidden_size).to(device)
dec = Decoder(dec_hidden_size, load_target_vocab.n_vocab).to(device) 193
train(load_pairs, load_source_vocab, load_target_vocab, enc, dec, 5000, print_every=1000)
evaluate(load_pairs, load_source_vocab, load_target_vocab, enc, dec, TARGET_MAX_LENGTH)
```


The seq2seq Chatbot using LSTM

- Data Preprocessing

```
9 torch.manual_seed(0)
10 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

12 raw = ["I feel hungry. 나는 배가 고프다.",
13        "Pytorch is very easy. 파이토치는 매우 쉽다.",
14        "Pytorch is a framework for deep learning. 파이토치는 딥러닝을 위한 프레임워크이다.",
15        "Pytorch is very clear to use. 파이토치는 사용하기 매우 직관적이다."] 16
17 SOS_token = 0
18 EOS_token = 1
19
20
```


The seq2seq Chatbot using LSTM

- Data Preprocessing

```
43 def preprocess(corpus, source_max_length, target_max_length):
44     print("reading corpus...")
45     pairs = []
46     for line in corpus:
47         pairs.append([s for s in line.strip().lower().split("\t")])
48     print("Read {} sentence pairs".format(len(pairs))) 49
50
51     pairs = [pair for pair in pairs if filter_pair(pair, source_max_length, target_max_length)] print("Trimmed to {}
52     sentence pairs".format(len(pairs)))
53     source_vocab = Vocab()
54     target_vocab = Vocab()
55     print("Counting words...") for pair in pairs:
56         source_vocab.add_vocab(pair[0])
57         target_vocab.add_vocab(pair[1])
58     print("source vocab size =", source_vocab.n_vocab)
59     print("target vocab size =", target_vocab.n_vocab)
60     return pairs, source_vocab, target_vocab
61
62
63
64
65
```

The seq2seq Chatbot using LSTM

- Models

```
66 class Encoder(nn.Module):
67     def __init__(self, input_size, hidden_size):
68         super(Encoder, self).__init__()
69         self.hidden_size = hidden_size
70         self.embedding = nn.Embedding(input_size, hidden_size)
71         self.gru = nn.GRU(hidden_size, hidden_size) 72
73
74     def forward(self, x, hidden):
75         x = self.embedding(x).view(1, 1, -
76         1) x, hidden = self.gru(x, hidden)
77         return x, hidden
78
```

The seq2seq Chatbot using LSTM

- Models

```
79 class Decoder(nn.Module):
80     def __init__(self, hidden_size, output_size):
81         super(Decoder, self).__init__()
82         self.hidden_size = hidden_size
83         self.embedding = nn.Embedding(output_size, hidden_size)
84         self.gru = nn.GRU(hidden_size, hidden_size)
85         self.out = nn.Linear(hidden_size, output_size)
86         self.softmax = nn.LogSoftmax(dim=1) 87
88
89
90     def forward(self, x, hidden):
91         x = self.embedding(x).view(1, 1, -1)
92         x, hidden = self.gru(x, hidden)
93         x = self.softmax(self.out(x[0]))
94         return x, hidden
```

Thank you